

Section D

Generative models: Restrictive Boltzmann Machines (RBMs), Introduction to MCMC and Gibbs Sampling, gradient computations in RBMs, Deep Boltzmann Machines.

Recent trends: Variational Autoencoders, Generative Adversarial Networks (GANs), Multitask Deep Learning, Multi-view Deep Learning

Applications: Vision, NLP, Speech (an overview of different applications)

What are Boltzmann Machines?

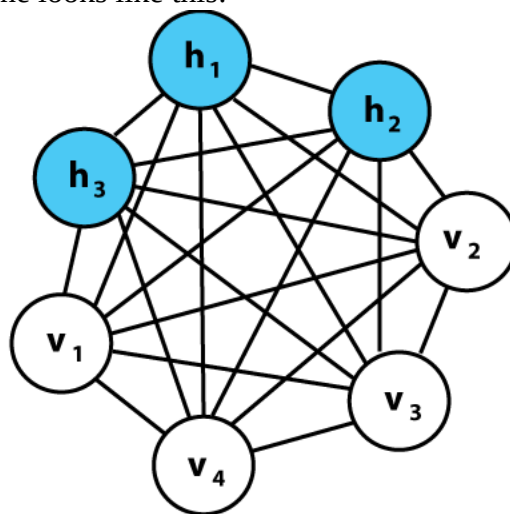
Boltzmann machines are stochastic and generative neural networks capable of learning internal representations, and are able to represent and (given sufficient time) solve difficult combinatoric problems.

They are named after the Boltzmann distribution (also known as Gibbs Distribution) which is an integral part of Statistical Mechanics and helps us to understand the impact of parameters like Entropy and Temperature on the Quantum States in Thermodynamics. That's why they are called Energy-Based Models (EBM). They were invented in 1985 by Geoffrey Hinton, then a Professor at Carnegie Mellon University, and Terry Sejnowski, then a Professor at Johns Hopkins University.

How do Boltzmann Machines work?

Boltzmann machines are non-deterministic (or stochastic) generative Deep Learning models with only two types of nodes - hidden and visible nodes. There are no output nodes! This may seem strange but this is what gives them this non-deterministic feature. They don't have the typical 1 or 0 type output through which patterns are learned and optimized using Stochastic Gradient Descent. They learn patterns without that capability and this is what makes them so special!

One difference to note here is that unlike the other traditional networks which don't have any connections between the input nodes, a Boltzmann Machine has connections among the input nodes. A Boltzmann Machine looks like this:



We can see from the image that all the nodes are connected to all other nodes irrespective of whether they are input or hidden nodes. This allows them to share information among themselves and self-generate subsequent data. We only measure what's on the visible nodes and not what's on the hidden nodes. When the input is provided, they are able to capture all

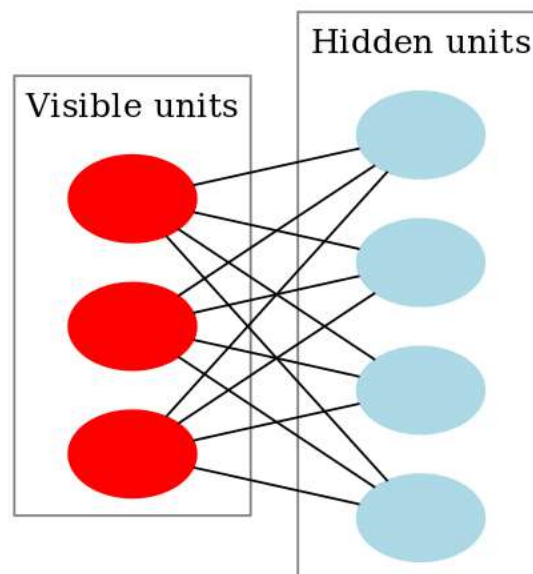
the parameters, patterns and correlations among the data. This is why they are called **Deep Generative Models** and fall into the class of Unsupervised Deep Learning.

What are Restricted Boltzmann Machines?

RBMs are a two-layered artificial neural network with generative capabilities. They have the ability to learn a probability distribution over its set of input. RBMs can be used for dimensionality reduction, classification, regression, collaborative filtering, feature learning, and topic modeling.

RBMs are a special class of Boltzmann Machines and they are restricted in terms of the connections between the visible and the hidden units. This makes it easy to implement them when compared to Boltzmann Machines. As stated earlier, they are a two-layered neural network (one being the visible layer and the other one being the hidden layer) and these two layers are connected by a fully bipartite graph. This means that every node in the visible layer is connected to every node in the hidden layer but no two nodes in the same group are connected to each other. This restriction allows for more efficient training algorithms than what is available for the general class of Boltzmann machines, in particular, the gradient-based contrastive divergence algorithm.

An RBM is in essence a two-layer neural network that is bidirectional and partially controlled by randomness. Also, the RBMs studied here have strictly binary activations of their hidden neurons, and binary inputs are favored. A Restricted Boltzmann Machine looks like this:



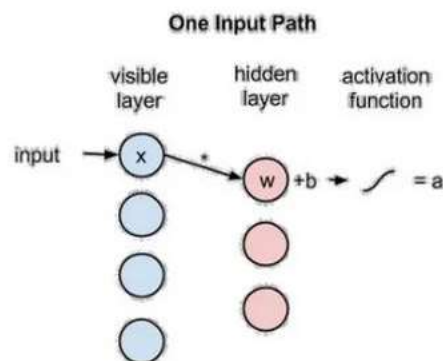
An RBM does not have an output layer as such. It has only an input layer (usually called the visible layer in the context of RBMs) and a hidden layer whose activations often serve as outputs.

Recall that for a traditional feedforward network, the activation of a neuron in a given layer is computed by forming the dot product of a weight vector with the activations of the prior layer, adding a bias term, and applying the logistic activation function. The same formulation applies to an RBM but with one exception. Instead of this procedure determining the activation of a hidden neuron given the inputs, it determines the probability that the hidden neuron will have the value one (as opposed to zero). Then the hidden neuron is randomly set to either zero or one according to this probability. RBMs are stochastic model, i.e., instead of calculating discrete values at each node the model calculates probabilities at each nodes and then the result passes through some activation function which gives output as 1 or 0.

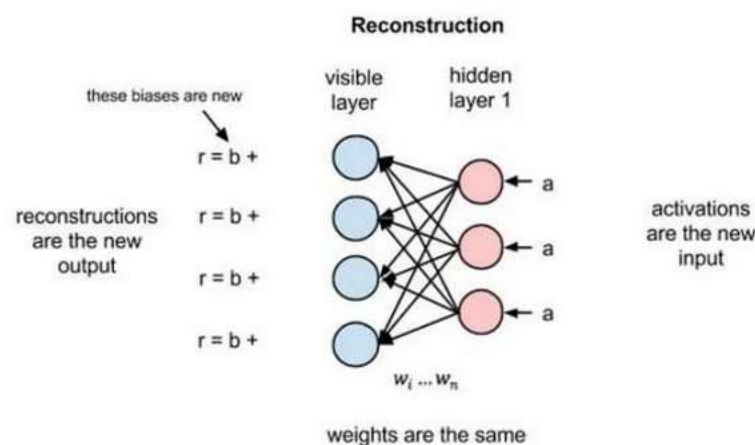
How do Restricted Boltzmann Machines work?

In an RBM, we have a symmetric bipartite graph where no two units within the same group are connected. Multiple RBMs can also be stacked and can be fine-tuned through the process of gradient descent and back-propagation. Such a network is called a Deep Belief Network. Although RBMs are occasionally used, most people in the deep-learning community have started replacing their use with General Adversarial Networks or Variational Autoencoders. This model contains two parts, i.e., forward pass and backward pass.

- In forward pass the models tries to calculate probabilities at each hidden neuron and decides whether to pass that input or not, and on backward pass the model tries to reconstruct input similar to the original input.



- The above image shows the first step in training an RBM with multiple inputs. The inputs are multiplied by the weights and then added to the bias. The result is then passed through a sigmoid activation function and the output determines if the hidden state gets activated or not. Weights will be a matrix with number of input nodes as the number of rows and number of hidden nodes as the number of columns. The first hidden node will receive the vector multiplication of the inputs multiplied by the first column of weights before the corresponding bias term is



- This is also known as one Gibbs sampling step
- One iteration of alternating Gibbs' sampling consists of – updating all hidden node in parallel followed by
 - updating all the visible nodes in parallel
- The hidden bias RBM produce the activation on the forward pass and the visible bias helps RBM to reconstruct the input during a backward pass.
- After a considerable number of back-and-forths, the sequence of states will converge to a stable Markov chain. The rate at which convergence is obtained is called the mixing rate and depends on W (smaller weights lead to faster mixing), but it is generally fast enough to be practical.

- At first the reconstructed inputs will be different from original input, but by changing biases and weights, errors can be reduced.
- Based on this learned weights and biases, when inputs are given new outputs can be predicted or reconstructed based on the given e.g. recommender system application. For example, we have a user's feedback on various movies but not all. We can predict missing feedback using RBM.

Gradient Computations:

The first step in training an RBM network is to come up with a set of weights and biases that will serve as an effective starting point for optimization using some gradient-based approach. The traditional method seems to be just generating a small random set of weights and then choosing initial bias vectors with a modest degree of intelligence. This may indeed be fine because the early stages of subsequent gradient-descent training rapidly improve the quality of the parameters. Since RBM operations are computationally expensive, it can be argued that any approach more sophisticated than a single initial random shot at starting weights is a waste of resources.

The idea behind stochastic gradient descent is that you compute an estimate of the gradient for some parameter set and then adjust the parameters by moving slightly in the direction opposite the gradient. At least for small moves, the error criterion will, on average, decrease. This simple algorithm is shown in Equations 1 and 2. In these equations, W_k is the weight set (including bias vectors) as of update k , $rate$ is a very small number called the learning rate, and Δ_k is the gradient at W_k .

$$increment_k = -rate * \Delta_k$$

(Eq 1)

$$W_{k+1} = W_k + increment_k$$

(Eq 2)

One problem with this simple approach is that when the error criterion function is a long, narrow gap, which happens often, the updates will bounce back and forth across the gap, making little net progress with each bounce. For this reason, a momentum term is usually included to propel adjustments in the average direction of movement. Each increment includes a fraction of the increment from the prior update, as shown in Equation 3. We can see the effect of momentum in Figure 3.2. Observe that without momentum the path would be perpendicular to the level curve. But by including an extra shove in the direction of the prior update, we push the net movement deeper into the hole.

$$increment_k = mom * increment_{k-1} - rate * \Delta_k$$

(Eq. 3)

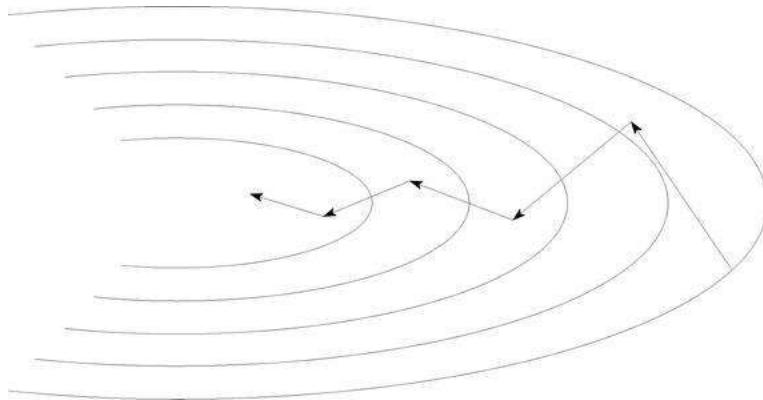
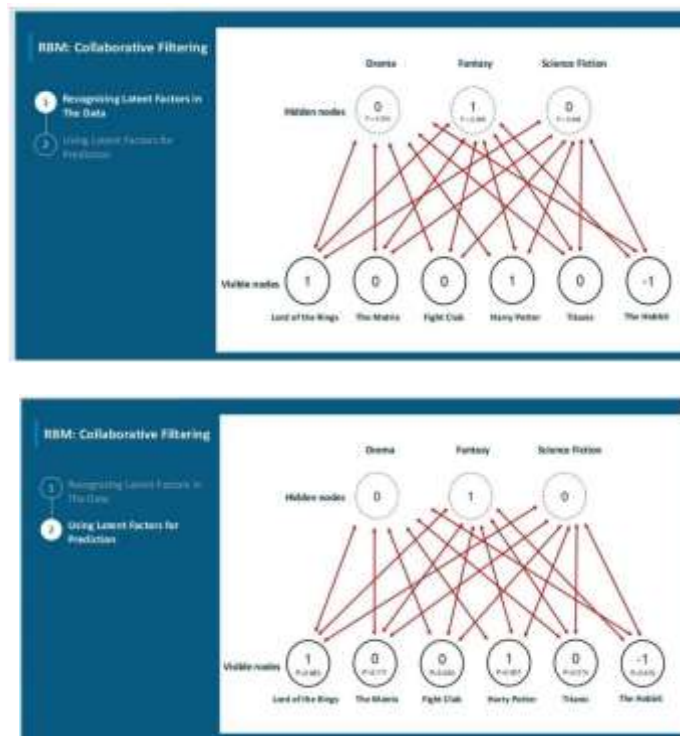


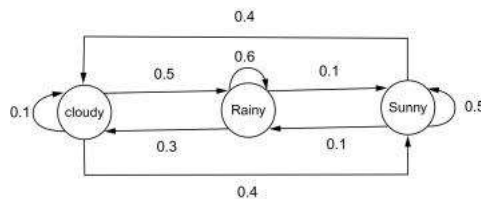
Figure 3-2: Momentum pushes rapidly into the center

RBM and Recommendation Systems



Introduction to MCMC and Gibbs Sampling

- Monte Carlo – Monte Carlo techniques just sample randomly and aggregate the results into an estimate of what's going to happen.
 - Simulation
 - Almost as soon as computers were invented, they were used for simulation
 - Monte Carlo at Los Alamos, one of the few places where computers were available at the time
 - the city in Monaco that has many casinos
 - Gambling was then (around 1950) illegal in most places, and the casino at Monte Carlo was the most famous in the world.
 - It soon became a colorless technical term for simulation of random processes.
- Markov Chain – A Markov Chain is simply a model of how your system moves from state to state.
 - Developing a Markov model from scratch can sometimes be difficult, but once you have one in hand, they're relatively easy to use, and relatively easy to understand.
 - The basic idea is that the system will have certain states associated with it; that as part of the system, it will move from state to state; and, critically, that this movement from state to state happens based on probability and that you know these probabilities.
 - Once you have that information, you can represent it all as a graph, where nodes are states and arcs between states (labelled with probabilities) are transitions.



- Markov chain is **a systematic method** for generating a sequence of random variables where the current value is probabilistically dependent on the value of the prior variable.
 - Specifically, selecting the next variable is only dependent upon the last variable in the chain.
- Consider a board game that involves rolling dice, such as snakes and ladders. • The roll of a die has a uniform probability distribution across 6 stages (integers 1 to 6). You have a position on the board, but your next position on the board is only based on the current position and the random roll of the dice. Your specific positions on the board form a Markov chain.
- The most popular method for sampling from high-dimensional distributions is Markov chain Monte Carlo or MCMC
- Specifically, MCMC is for performing inference (e.g. estimating a quantity or a density) for probability distributions where independent samples from the distribution cannot be drawn, or cannot be drawn easily.

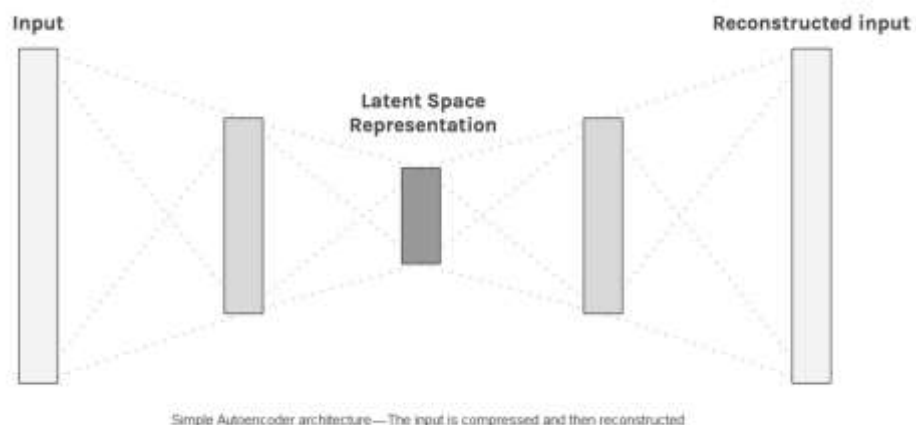
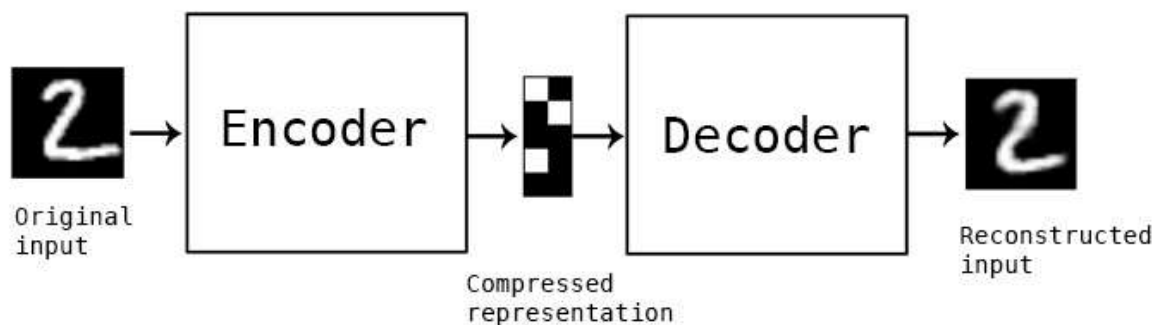
- As such, Monte Carlo sampling cannot be used. Instead, samples are drawn from the probability distribution by constructing a Markov Chain, where the next sample that is drawn from the probability distribution is dependent upon the last sample that was drawn.
- the idea of imposing a dependency between samples may seem odd at first, but may make more sense if we consider domains like the snakes and ladders games, where such dependency between samples is required.
- Unlike Monte Carlo sampling methods that are able to draw independent samples from the distribution, Markov Chain Monte Carlo methods draw samples where the next sample is dependent on the existing sample, called a Markov Chain.
- This allows the algorithms to narrow in on the quantity that is being approximated from the distribution, even with a large number of random variables.
- The most common general Markov Chain Monte Carlo algorithm is called Gibbs Sampling
- The Gibbs Sampling is a Monte Carlo Markov Chain method that iteratively draws an instance from the distribution of each variable, conditional on the current values of the other variables in order to estimate complex joint distributions.
- The Gibbs Sampling algorithm is an **approach to constructing a Markov chain** where the probability of the next sample is calculated as the conditional probability given the prior sample. $P(X,Y)$.
- Suppose $p(x, y)$ is a probability distribution that is difficult to sample from directly. Suppose, though, that we can easily sample from the conditional distributions $p(x|y)$ and $p(y|x)$.
- The Gibbs sampling algorithm proceeds as follows:
 - set x and y to some initial starting values (x_0, y_0)
 - then sample $x|y$, then sample $y|x$, then $x|y$, and so on.
 - Sample $x_1 \sim p(x|y_0)$, that is, from the conditional distribution $X | Y = y_0$. Current state: (x_1, y_0)
 - Sample $y_1 \sim p(y|x_1)$, that is, from the conditional distribution $Y | X = x_1$. Current state: (x_1, y_1)
 - Sample $x_2 \sim p(x|y_1)$, that is, from the conditional distribution $X | Y = y_1$. Current state: (x_2, y_1)
 - Sample $y_2 \sim p(y|x_2)$, that is, from the conditional distribution $Y | X = x_2$. Current state: (x_2, y_2)
 - . . . Repeat iterations, M times.

This procedure defines a sequence of pairs of random variables $(X_0, Y_0), (X_1, Y_1), (X_2, Y_2), (X_3, Y_3), \dots$ satisfying the property of being a Markov chain

AUTOENCODERS

Neural networks exist in all shapes and sizes, and are often characterized by their **input** and **output data type**. For instance, image classifiers are built with *Convolutional Neural Networks*. They take **images** as inputs, and output a **probability distribution of the classes**.

Autoencoders (AE) are a family of neural networks for which **the input is the same as the output***. They work by compressing the input into a *latent-space representation*, and then reconstructing the output from this representation.



- **BUT WHY?**
 - Useful when the data is not labelled
 - A workaround to handle unlabelled data
- A typical use of a **Neural Network** is a case of supervised learning.
 - It involves training data which **contains an output label**. The neural network tries to learn the mapping from the given input to the given output label.

Input

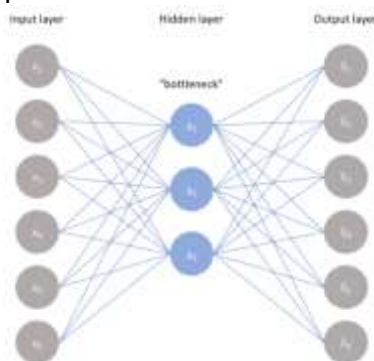
output

	A	B	C	D	E
1	sepal_len	sepal_wid	petal_len	petal_wid	Species
2	5.1	3.5	1.4	0.2	setosa
3	4.9	3	1.4	0.2	setosa
4	4.7	3.2	1.3	0.2	setosa
5	4.6	3.1	1.5	0.2	setosa
6	5	3.6	1.4	0.2	setosa
7	5.4	3.9	1.7	0.4	setosa
8	4.6	3.4	1.4	0.3	setosa
9	5	3.4	1.5	0.2	setosa
10	4.4	2.9	1.4	0.2	setosa
11	4.9	3.1	1.5	0.1	setosa
12	5.4	3.7	1.5	0.2	setosa
13	4.8	2.8	1.6	0.2	setosa

- But what if the output label is replaced by the input vector itself?
 - Then the network will try to find the mapping from the input to itself. This would be the identity function **which is a trivial 1-1 mapping**. So network will hardly learn anything.

Input					output				
	A	B	C	D		A	B	C	D
1	sepal_len	sepal_wid	petal_len	petal_wid	1	sepal_len	sepal_wid	petal_len	petal_wid
2	5.1	3.5	1.4	0.2	2	5.1	3.5	1.4	0.2
3	4.9	3	1.4	0.2	3	4.9	3	1.4	0.2
4	4.7	3.2	1.3	0.2	4	4.7	3.2	1.3	0.2
5	4.6	3.1	1.5	0.2	5	4.6	3.1	1.5	0.2
6	5	3.6	1.4	0.2	6	5	3.6	1.4	0.2
7	5.4	3.9	1.7	0.4	7	5.4	3.9	1.7	0.4
8	4.6	3.4	1.4	0.3	8	4.6	3.4	1.4	0.3
9	5	3.4	1.5	0.2	9	5	3.4	1.5	0.2
10	4.4	2.9	1.4	0.2	10	4.4	2.9	1.4	0.2
11	4.9	3.1	1.5	0.1	11	4.9	3.1	1.5	0.1
12	5.4	3.7	1.5	0.2	12	5.4	3.7	1.5	0.2
13	4.8	2.8	1.6	0.2	13	4.8	2.8	1.6	0.2

- But if the network is not allowed to simply copy the input, then the network will be forced to capture only the salient features.
- Autoencoders are an **unsupervised learning technique** in which we leverage neural networks for the task of **representation learning**.
 - The network is first trained on the given input.
 - The network tries to reconstruct the given input from the features it picked up and gives an approximation to the input as the output.
 - The training step involves the computation of the error and backpropagating the error.
 - The typical architecture of an Auto-encoder resembles a bottleneck.

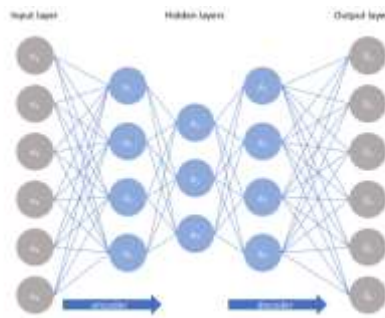


- So, we can take an unlabeled dataset and frame it as a supervised learning problem tasked with outputting $x^\wedge$, a **reconstruction of the original input x** .
- The bottleneck is a key attribute of our network design; without the presence of an information bottleneck, our network could easily learn to simply memorize the input values by passing these values along through the network (visualized below).
 - A bottleneck constrains the amount of information that can traverse the full network, forcing a learned compression of the input data.
- The ideal autoencoder model balances the following:
 - Sensitive to the inputs enough to accurately build a reconstruction.
 - Insensitive enough to the inputs that the model doesn't simply memorize or overfit the training data.
- This trade-off forces the model to maintain only the variations in the data required to reconstruct the input without holding on to redundancies within the input.
 - For most cases, this involves constructing a loss function where one term encourages our model to be sensitive to the inputs (ie. reconstruction loss $L(x, x')$) and a second term discourages memorization/overfitting (ie. an added regularizer).

$$\mathcal{L}(x, \hat{x}) + \text{regularizer}$$

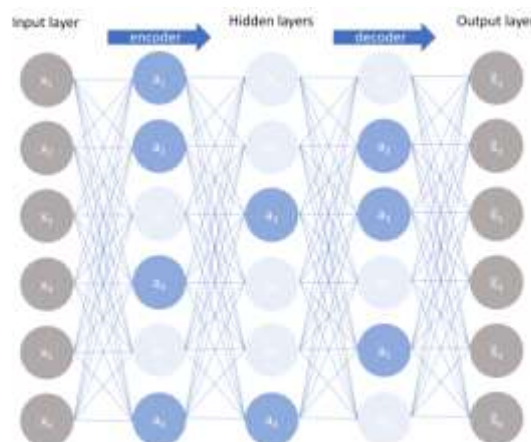
- We'll typically add a scaling parameter in front of the regularization term so that we can adjust the trade-off between the two objectives.
- Some of the standard autoencoder architectures for imposing these two constraints and tuning the trade-offs are as follows;
 - An undercomplete Auto Encoder
 - The simplest architecture for constructing an autoencoder is to restrict the number of nodes in the hidden layer(s) of the network, limiting the amount of information that can flow through the network.
 - By penalizing the network according to the reconstruction error, our model can learn the most important attributes of the input data and how to best reconstruct the original input from an "encoded" state.
 - Encoding is achieved by the encoder part of the network which has **decreasing number of hidden units** in each layer.
 - Thus this part is forced to pick up only the most significant and representative features of the data. It encodes the data using the initialized weights and biases.
 - Ideally, this encoding will **learn and describe latent (a.k.a. hidden) attributes of the input data**.
 - The second half of the network performs the **Decoding function**. This part has **the increasing number of hidden units** in each layer and thus tries to reconstruct the original input from the encoded data.
 - After the reconstruction, the loss function is computed to determine the reliability of the encoding. The error generated is backpropagated.
 - The above-described training process is reiterated several times until an acceptable level of reconstruction is reached.

- After the training process, only the encoder part of the Auto-encoder is retained to encode a similar type of data used in the training process.
- An undercomplete autoencoder has no explicit regularization term - we simply train our model according to the reconstruction loss. Thus, our only way to ensure that the model isn't memorizing the input data is to ensure that we've sufficiently restricted the number of nodes in the hidden layer(s).



○ Sparse autoencoders

- Sparse autoencoders offer us an alternative method for introducing an information bottleneck without *requiring* a reduction in the number of nodes at our hidden layers. Rather, we'll construct our loss function such that we penalize *activations* within a layer. For any given observation, we'll encourage our network to learn an encoding and decoding which only relies on activating a small number of neurons. It's worth noting that this is a different approach towards regularization, as we normally regularize the *weights* of a network, not the activations.
- A generic sparse autoencoder is visualized below where the opacity of a node corresponds with the level of activation. It's important to note that the individual nodes of a trained model which activate are *data-dependent*, different inputs will result in activations of different nodes through the network.



- One result of this fact is that **we allow our network to sensitize individual hidden layer nodes toward specific attributes of the input data**. Whereas an undercomplete autoencoder will use the entire

network for every observation, a sparse autoencoder will be forced to selectively activate regions of the network depending on the input data. As a result, we've limited the network's capacity to memorize the input data without limiting the network's capability to extract features from the data.

- So, the different ways to constrain the network to create an autoencoder neural network are:-
 - **Keep small Hidden Layers:** If the size of each hidden layer is kept as small as possible, then the network will be forced to pick up only the representative features of the data thus encoding the data.
 - **Tuning the Activation Functions:** This method involves changing the activation functions of various nodes so that a majority of the nodes are dormant thus effectively reducing the size of the hidden layers.
 - **Regularization:** In this method, **a loss term is added** to the cost function which encourages the network to train in ways other than copying the input.
 - **Denoising:** Another way of constraining the network is to **add noise to the input** and teaching the network how to remove the noise from the data.
- The different variations of Auto-encoders are
 - **Undercomplete Auto Encoder:** The simplest architecture for constructing an autoencoder is to constrain the number of nodes present in the hidden layer(s) of the network, limiting the amount of information that can flow through the network.
 - See an implementation on this link:
 - https://colab.research.google.com/drive/1xexA17vMz_CjaiEsKObGU_NYFS3-V4gMK?usp=sharing
 - **Sparse Auto-encoder:** This type of auto-encoder typically contains more hidden units than the input but only a few are allowed to be active at once. This property is called the sparsity of the network. The sparsity of the network can be controlled by either manually zeroing the required hidden units, tuning the activation functions or by adding a loss term to the cost function.
 - **Denoising Auto-encoder:** This type of auto-encoder works on a partially corrupted input and trains to recover the original undistorted image. As mentioned above, this method is an effective way to constrain the network from simply copying the input.
 - **Variational Auto-encoder:** This type of auto-encoder makes strong assumptions about the distribution of latent variables and uses the **Stochastic Gradient Variational Bayes** estimator in the training process.
- **Variational autoencoder (VAE)**
 - Variational autoencoders are a slightly more modern and interesting take on autoencoding.
 - What is a variational autoencoder?

- It's a type of autoencoder with added constraints on the encoded representations being learned.
- More precisely, it is an autoencoder that learns a [latent variable model](#) for its input data.
 - So **instead of letting the neural network learn an arbitrary function**, it is **learning the parameters of a probability distribution modelling the data**.
 - If you sample points from this distribution, you can generate new input data samples: a VAE is a "generative model".
- How does a variational autoencoder work?
 - First, an encoder network turns the input samples x into two parameters in a latent space, which we will note z_mean and z_log_sigma .
 - Then, we randomly sample similar points z from the latent normal distribution that is assumed to generate the data, via $z = z_mean + \exp(z_log_sigma) * \epsilon$, where ϵ is a random normal tensor.
 - Finally, a decoder network maps these latent space points back to the original input data.
- The parameters of the model are trained via two loss functions:
 - a reconstruction loss forcing the decoded samples to match the initial inputs (just like in the undercomplete autoencoder), and
 - the KL divergence between the learned latent distribution and the prior distribution, acting as a regularization term.

Generative Adversarial Networks (GANs)

GANs are generative models: they create new data instances that resemble your training data. For example, GANs can create images that look like photographs of human faces, even though the faces don't belong to any real person.

These images were created by a GAN:



Figure 1: Images generated by a GAN created by NVIDIA.

GANs achieve this level of realism by pairing a generator, which learns to produce the target output, with a discriminator, which learns to distinguish true data from the output of the generator. The generator tries to fool the discriminator, and the discriminator tries to keep from being fooled.

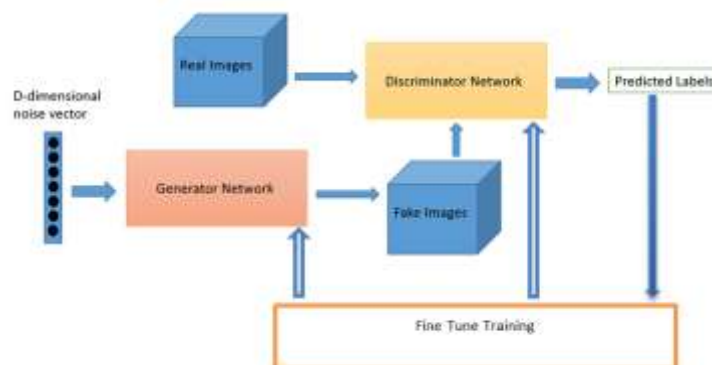
- Generative modeling is an **unsupervised learning task** in machine learning that **involves automatically discovering and learning the regularities or patterns in input data in such a way that the model can be used to generate or output new examples that plausibly could have been drawn from the original dataset.**
- GANs are a **clever way** of training a generative model by framing the problem as a supervised learning problem with two adversarial (or conflicting) sub-models:
 - the generator model that we train to generate new examples, and
 - the discriminator model that tries to classify examples as either real (from the domain) or fake (generated).
 - The two models are trained together in a zero-sum game, adversarial, until the discriminator model is fooled about half the time, meaning the generator model is generating plausible examples.
- **Generative vs. Discriminative Algorithms**
 - How discriminative algorithms work?
 - Discriminative algorithms try to classify input data; that is, given the features of an instance of data, they predict a label or category to which that data belongs.
 - For example, given all the words in an email (the data instance), a discriminative algorithm could predict whether the message is spam or not_spam. spam is one of the labels, and the bag of words gathered from the email are the features that constitute the input data. When this problem is expressed mathematically, the label is called y and the features are called

x. The formulation $p(y|x)$ is used to mean “the probability of y given x”, which in this case would translate to “the probability that an email is spam given the words it contains.”

- So discriminative algorithms map features to labels. They are concerned solely with that correlation.
- The generative algorithms do the opposite. Instead of predicting a label given certain features, they attempt to predict features given a certain label.
 - The question a generative algorithm tries to answer is: Assuming this email is spam, how likely are these features?
 - While discriminative models care about the relation between y and x, generative models care about “how you get x?”
 - They allow you to capture $p(x|y)$, the probability of x given y, or the probability of features given a label or category.

- **How GANs Work**

- One neural network, called the *generator*, generates new data instances, while the other, the *discriminator*, evaluates them for authenticity; i.e. the discriminator decides whether each instance of data that it reviews belongs to the actual training dataset or not.
- For example, for generating human faces, generator creates images which look like human faces. The goal of the discriminator, when shown an instance from the true faces dataset from the real world, is to recognize those that are authentic.
- Here are the steps a GAN takes:
 - The generator takes in random numbers and returns an image.
 - This generated image is fed into the discriminator alongside a stream of images taken from the actual, ground-truth dataset.
 - The discriminator takes in both real and fake images and returns probabilities, a number between 0 and 1, with 1 representing a prediction of authenticity and 0 representing fake.



- After the training process, the discriminator model is discarded as we are interested in the generator.

- **Example**

- When training begins, the generator produces obviously fake data, and the discriminator quickly learns to tell that it's fake:



- As training progresses, the generator gets closer to producing output that can fool the discriminator:



- Finally, if generator training goes well, the discriminator gets worse at telling the difference between real and fake. It starts to classify fake data as real, and its accuracy decreases.



- GANs take a long time to train. On a single GPU a GAN might take hours, and on a single CPU more than a day. While difficult to tune and therefore to use, GANs have stimulated a lot of interesting research and writing.

Multi-task learning

Multi-task learning (MTL) is a model training technique where you train a single deep neural network on multiple tasks at the same time. Though it may seem a little counter-intuitive at first — shouldn't learning two problems be harder than learning one? — MTL turns out to often improve performance of the final model compared to a single-task variant, which is why it's becoming more and more popular in deep learning.

Multi-Task Learning resembles the mechanism of human learning more closely than Single-Task Learning because we humans often learn transferable skills. For example, learning to ride a bicycle makes it easy for someone to learn to ride a motorbike later on, which builds upon similar concepts of body balance. Moreover, learning to ride a motorcycle with gears helps to learn to drive a car with manual transmission faster. This is referred to as the inductive transfer of knowledge. This mechanism of knowledge transfer is what allows humans to learn new concepts with only a few examples or no examples at all.

Multi-task learning is a training paradigm in which machine learning models are trained with data from multiple tasks simultaneously, using shared representations to learn the common ideas between a collection of related tasks. These shared representations increase data efficiency and can potentially yield faster learning speed for related or downstream tasks, helping to alleviate the well-known weaknesses of deep learning: large-scale data requirements and computational demand. However, achieving such effects has not proven easy and is an active area of research today.

Single Task Learning Figure 1 shows four separate artificial neural networks (ANNs). Each net is a function of the same inputs, and has one output. Backpropagation is applied to these nets by training each net in isolation. Because the four nets are not connected, it is not possible for what one net learns to help another net. We call this approach Single Task Learning (STL).

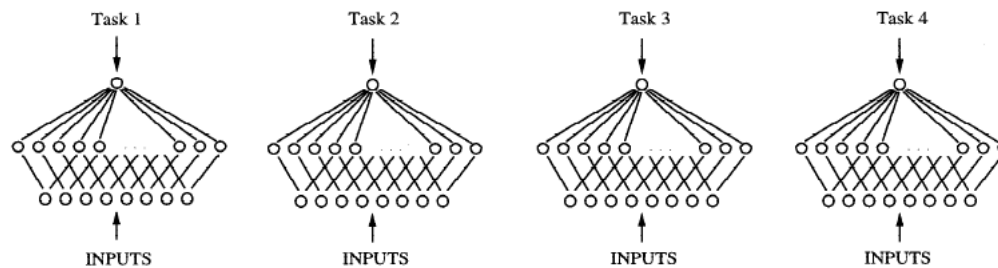


Figure 1. Single Task Learning (STL) of four tasks with the same inputs

Multi Task Learning

Figure 2 shows a single net with the same inputs as the four nets in Figure 1, but which has four outputs, one for each task the nets in Figure 1 were being trained on. Note that these four outputs are fully connected to a hidden layer that they share.¹ Backpropagation is done in parallel on the four outputs in the MTL net. Because the four outputs share a common hidden layer, it is possible for internal representations that arise in the hidden layer for one task to be used by other tasks. Sharing what is learned by different tasks while tasks are trained in parallel is the central idea in multitask learning.

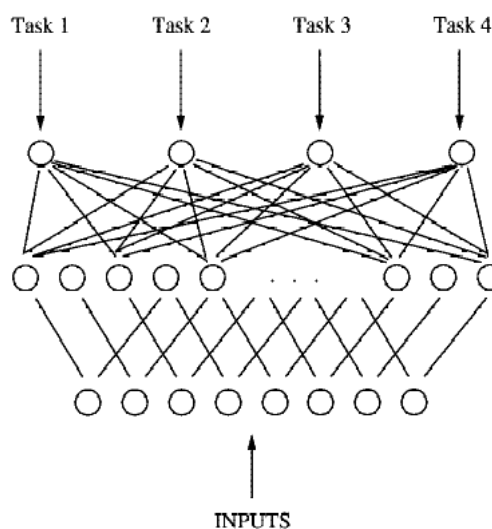


Figure 2. Multitask Learning (MTL) of four tasks with the same inputs.

Benefits of MTL:

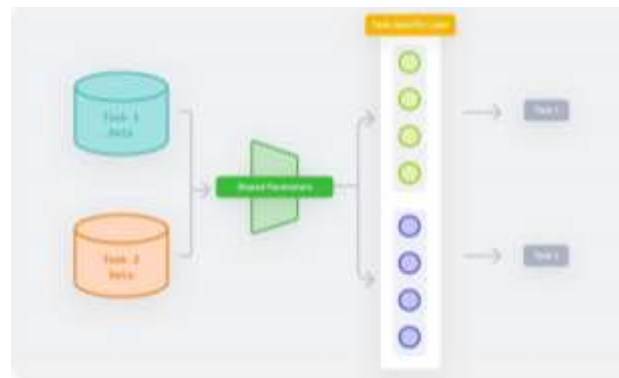
- **Improved Generalization:** MTL can enhance model generalization by leveraging shared knowledge across tasks, leading to better performance on unseen data.
- **Data Efficiency:** MTL can exploit data from related tasks to improve performance, especially in scenarios with limited labeled data for individual tasks.
- **Regularization:** Sharing information across tasks can act as a form of regularization, reducing overfitting and improving model robustness.

Multi-Task Learning Architectures

There are many different factors to consider when creating a shared architecture, such as the portion of the model's parameters that will be shared between tasks, and how to parameterize and combine task-specific and shared modules.

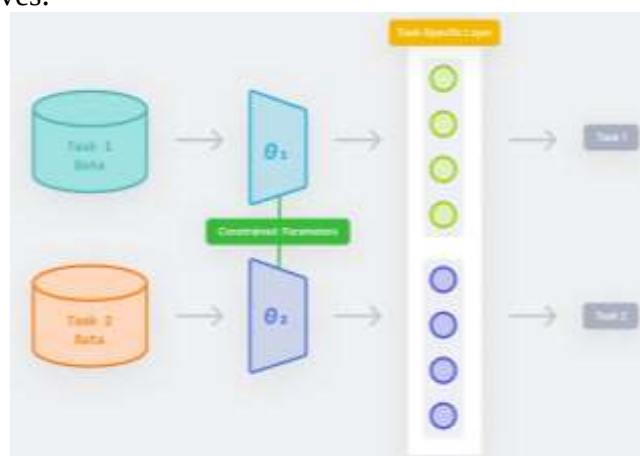
Hard Parameter Sharing: Shared layers are used across all tasks, promoting weight sharing and facilitating knowledge transfer. Examples include convolutional neural networks (CNNs) with shared convolutional layers for image-related tasks.

In Hard Parameter Sharing, the hidden layers of the neural networks are shared while keeping some task-specific output layers. Sharing most of the layers for the related tasks reduces the chances of overfitting. The more tasks a shared model is learning simultaneously, the more it has to find a representation that captures all of the tasks, and the smaller the chance of overfitting on the original task.



Soft Parameter Sharing: Hard parameter sharing performs well only if tasks are closely related. Therefore, new approaches have focused on learning the features that need to be shared between tasks. Regularization has long played an important role in multi-task learning, mostly in the form of soft parameter sharing. Soft parameter sharing is one of two popular techniques for MTL (the other being hard parameter sharing) in which parameters aren't shared between task models, but instead the L2 distance between the parameters of task models is added to the training objective, in order to encourage similar model parameters between different tasks. Soft parameter sharing is simple to implement and has been employed extensively in MTL methods.

Shared layers are encouraged to learn similar representations across tasks, but with flexibility to adapt to task-specific features. This is often achieved through regularization techniques or auxiliary task objectives.

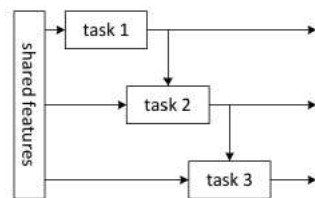


Cross-Stitch Networks: Introduces learnable parameters to dynamically control the information flow between shared and task-specific layers, allowing for adaptive sharing of features based on task requirements. The Cross-Stitch Network automatically learns an optimal combination of shared and task-specific representations. It models shared

representations using linear combinations and learns the optimal linear combinations for a given set of tasks.

A large portion of the MTL literature is devoted to the design of multi-task neural network architectures. There are many different factors to consider when creating a shared architecture, such as the portion of the model's parameters that will be shared between tasks, and how to parameterize and combine task-specific and shared modules. More variations arise when considering architectures for a specific problem domain, like how to partition convolutional filters into shared and task-specific groups for a set of vision tasks. Many of the proposed architectures for MTL play a balancing game with the degree of information sharing between tasks: Too much sharing will lead to negative transfer and can cause worse performance of joint multi-task models than individual models for each task, while too little sharing doesn't allow the model to effectively leverage information between tasks. The best performing architectures for MTL are those which balance sharing well.

In Multi-task Network Cascades, the output of the first task is used as an input for the second task, the second task's output is used as an input for the third task, and so on.



Optimization for Multi-Task Learning

With MTL architecture design as the modern generalization of hard parameter sharing on one side, MTL optimization is the broader version of soft parameter sharing on the other. Soft parameter sharing is a way to regularize model parameters by penalizing the distance from model parameters to corresponding parameters of a model for a different, but related task.

Loss Weighting

A very common approach to ease multi-task optimization is to balance the individual loss functions for different tasks. When a model is to be trained on more than one task, the various task-specific loss functions must be combined into a single aggregated loss function which the model is trained to minimize. A natural question to ask then, is how to exactly combine multiple loss functions into one that is suitable for MTL. Most of the methods parameterize the aggregated loss function as a weighted sum of the task-specific loss functions.

Task Scheduling

Most Multi-Task Learning models make a decision on which task(s) to train on in an epoch in a very simple way, either training on all tasks at each step or randomly sampling a subset of tasks to train on. However, intelligently optimized task scheduling can significantly improve the overall model performance on all tasks.

For example, in a multi-task multi-lingual model where tasks are scheduled according to the similarity between each task and the primary task, it considers both task similarity and the number of training samples available for the task to compute the choice for tasks to be trained in each step.

Negative Transfer

Most Multi-Task Learning approaches assume that the individual tasks used for the joint optimization are closely related. However, each task might not be closely related to all available tasks. In such cases, sharing information with an unrelated task might even hurt performance, a phenomenon known as “negative transfer.”

From an optimization perspective, negative transfer manifests as the presence of conflicting task gradients. When two tasks have gradient vectors that point in opposing directions, following the gradient for one task will decrease the performance of the other task. Following the average of the two gradients means that neither task sees the same improvement as in a single-task training setting. Thus, modulation of task gradients is a potential solution to this problem.

Multi-view learning

Multi-view learning is concerned with the problem of machine learning from data represented by multiple distinct feature sets. The recent emergence of this learning mechanism is largely motivated by the property of data from real applications where examples are described by different feature sets or different “views”. For instance, in multimedia content understanding, multimedia segments can be simultaneously described by their video and audio signals. In web-page classification, a web page can be described by the document text itself and at the same time by the anchor text attached to hyperlinks pointing to this page. As another example, in content-based web-image retrieval, an object is simultaneously described by visual features from the image and the text surrounding the image.

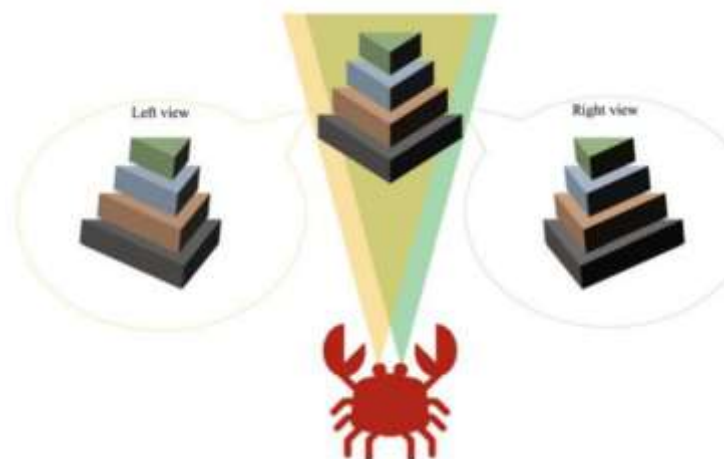


Fig: Each eye captures its own view and the two views are sent on to the brain for processing

In many scientific data analysis tasks, data are often collected through different measuring methods, such as various feature extractors or sensors, as usually the single particular measuring method cannot comprehensively describe all the information of the data. In this case, the features of each data example can be naturally divided into groups, each of which can be considered as a view. For instance, for images and videos, color features and texture features can be regarded as two views. It is important to make good use of the information from different views.

Multiview machine learning (referred to as multiview learning for short) is a branch of machine learning which studies and utilizes the information and relationship between views.

The complementarity from different representative views can effectively improve the performance of artificial intelligence systems. In the natural language processing problem, it is very beneficial for many sequence annotation tasks if you have access to the following context in addition to the proceeding one. In the case of special character classification, it would be very helpful that you know the next letter as you know the previous letter. For example, bi-directional long short-term memory (BiLSTM) not only considers the information contained in the sequence, but also consider the information of the reverse sequence as the second representation to complement the comprehensive description of the sequence. Similarly, animals with two eyes (two views) can capture depth information in three dimensional that cannot be captured by a single view as depicted.

How to better utilize the relationships between views is another important aspect. In order to adapt the traditional single-view learning algorithms, one approach can be simply concatenating all multiple views into one single view. But there are two shortcomings: it may break the balance between the size and the dimensions of the data and cause over-fitting; it may ignore the obvious differences in the physical interpretations of different views. Multiview learning explicitly models the relationship between these different views and/or the results from downstream computation.

It better utilizes the structural information in the data, which leads to better modeling capability. Further, multiple views can act as complementary and collaborative normalization, effectively reducing the size of the hypothesis space. Moreover, a noteworthy fact for multi-view learning is that when a natural feature split does not exist, performance improvements can still be observed using manufactured splits. Therefore, multi-view learning is a very promising topic with widespread applicability.

Four typical application areas as follows:

Cross-Media Intelligence.

Information retrieval is a process of finding desired or interesting contents from a collection of information. Various multimedia information (such as images, texts, and videos, etc.) coexisting on the internet requires crossmedia retrieval techniques. The key of cross-media retrieval techniques is modelling relationships between different media representations and reducing the semantic gaps to achieve the purpose of using a modal example to retrieve other modal examples with similar semantics. Generating text from images and videos, or the inverse is important to research, where the generating between text and videos is a multiview learning problem with sequential representations. Another research direction in cross-media intelligence technology is video classification. Since the representation of a video usually involves a variety of features (such as image frame, image stream, audio, and text, etc.), video classification is a typical multiview learning problem.

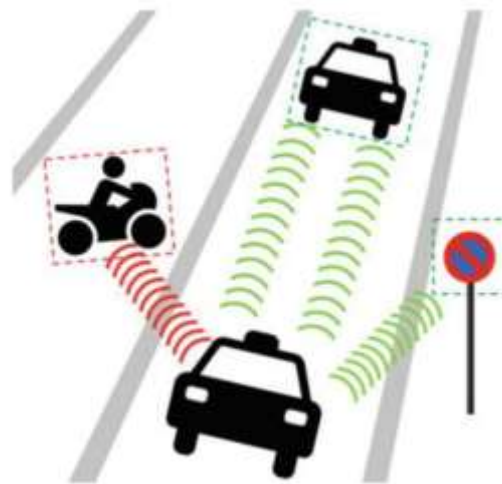
Intelligent Medical.

The development of artificial intelligence has provided very advantageous technical support for the intelligent development in the medical and health field. Intelligent medical plays an important role in computer-aided diagnosis, medical image analysis, and disease prediction. Sampling patient data often involves multiple sets of different tests, and doctors make a

diagnosis based on these results. When diagnosing images of pathological slices, pathologists often combine text (natural language) descriptions of the patient's condition from clinicians. In addition, there are also a large number of sequential data in the medical and health field, such as the patient's electronic health records, which constitutes multiview sequential data. Modeling the dynamic of people's health records to reveal the principles underlying the changes in patients' conditions over time can provide theoretical and algorithmic foundations for the development of precision medicine.

Automatic Driving.

Automatic driving relies on the collaboration of artificial intelligence, computer vision, radar, GPS, and so on, so that the computer can automatically control the vehicle safely without human initiative in Fig.



When the automatic driving system perceives the environment, it often depends on the multiple types of equipment such as radar, camera, GPS, and so on. These equipment provide different representations of the environment that a vehicle faces.

Intelligent Education.

Education takes an important strategic position in the development of a country. The application of artificial intelligence technology to analyze educational data can promote personalized education, campus safety, and a lifelong learning, etc. Another application of artificial intelligence technology is information recommendation, where content may be recommended to specific groups through a trained classifier according to the different learning stages of the involved students. In addition to the text (natural language), the content is often accompanied by rich images, which form two different representations of the content.